

Overview of SSL/TLS protocols

The SSL protocol comprises the following steps:

1. Client establishes a TCP connection with server.
2. A SSL handshake takes place.
 - a. Client sends a ClientHello message that indicates:
 - The max SSL/TLS protocol version that the client can support.
 - Symmetric and asymmetric ciphers that the client supports. The complete cipher string is written as: `<protocol>_<key_exchange_scheme>_<certificate>_<symmetric_cipher>_<hash_algo>`.
For example, `TLS_RSA_WITH_AES_128_CBC_SHA`, indicates use of RSA for key exchange and certificate, with AES-128 and SHA hashing algorithm. Another example is `ECDHE-RSA-AES128-SHA256`, where the key exchange is Elliptic Curve Diffie-Hellman, the certificate is of type RSA, AES128 is the symmetric cipher and SHA256 is the hashing algorithm.
 - b. The server picks a suitable protocol version, less than or equal to the maximum given by the client. It also picks the cipher string.
 - c. Then the server sends its certificate to the client. The client authenticates the server by verifying the certificate. This requires the client to follow the chain of trust of the given certificate, which finally leads to a root that is already trusted by the client.
 - d. In the next step (assuming RSA key exchange), the client generates a pre-master secret and encrypts it with the public key from the server's certificate. This is sent in the ClientKeyExchange message.
 - e. As the server has the private key for the certificate that it shared, it can decrypt the pre-master secret from the ClientKeyExchange message.
 - f. Then both parties compute the session key from the pre-master secret, using a standardised algorithm.
 - g. Once the session key is

computed, the parties send the ChangeCipherSpec message, indicating that subsequent messages will be encrypted. This ends the handshake.

3. The next step is the SSL Record protocol.
 - a. The Record protocol applies when the parties share a session key, either through an immediately preceding handshake or by resuming an earlier session.
 - b. In either case, the Record protocol takes the application data and carries out the following steps:
 - Fragments it
 - May optionally compress the fragments
 - Adds a hash (called MAC or Message Authentication Code) to each fragment
 - Encrypts the hash-appended fragments
 - Prepends the SSL Record header
 - And finally transmits the fragment
- Thus, the protocol ensures the following:
1. Server authentication, by verifying the server certificate
 2. Secure key exchange leading to a common session key for confidentiality
 3. Message hashing for integrity

TLS 1.2 improvements

While TLS 1.2 has fundamentally the same flow as its SSL and TLS predecessors, the protocol has been enhanced in the following ways.

1. *Addition of support for authenticated encryption (AE)*: This provides protection from chosen ciphertext attacks.

2. *Removal of MD5 hashing scheme*: MD5 is considered cryptographically broken and was exploited by the Flame malware in 2012. Almost seven years earlier, a team of researchers demonstrated the construction of two X509 certificates with different public keys and with the same MD5 hash, thereby showing a practical case of hash collision.
3. Support for SSLv2 Hello has been removed from the default options. There are also other improvements but all of them fall in the category of tightening the protocol. However, the protocol changed in the next version, TLS 1.3, which we will cover in the next article in this series.

In this article, we started with understanding the characteristics of modern Web portals, in terms of the number of connections and the number of HTTP requests required to render the front page. These metrics indicate that the quantum of traffic is quite high, and the limitations of the HTTP 1.1 protocol can impact the latency seen by the end user. This led us to the HTTP/2 protocol, from which we understood how concurrent, multi-streams were introduced to address the protocol level inefficiencies. The HTTP/2 protocol is widely supported only over a TLS secure channel. Then, we switched gears and understood the operation of the SSL/TLS protocol and enumerated the security enhancements made in the TLS 1.2 protocol, which enabled it to be ahead of many of the security vulnerabilities found in earlier versions of the protocol. **END** 🐧

References

- [1] HTTP/2: <https://tools.ietf.org/rfc/rfc7540.txt>
- [2] HPACK – HTTP Header Compression: <https://tools.ietf.org/rfc/rfc7541.txt>
- [3] TLS 1.2: <https://tools.ietf.org/rfc/rfc5246.txt>
- [4] SSL/TLS protocol vulnerabilities: <https://www.acunetix.com/blog/articles/tls-vulnerabilities-attacks-final-part/>

By: Sriram Gorti

The author is an avid fan of Python for its simplicity and ease of programming, and tinkers with it on weekends. On weekdays, he maintains and extends production code for a cloud Web security solution.